

Computer Vision Pipeline Specification

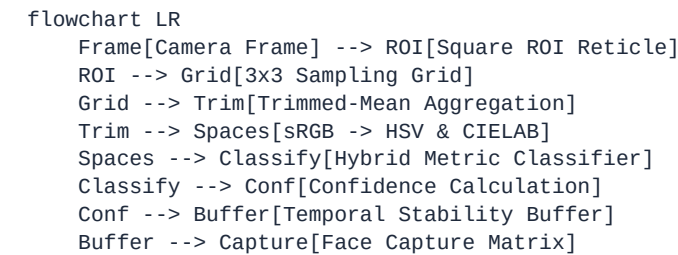
Source: docs/COMPUTER_VISION.md • Status: Verified & Synchronized

Computer Vision Pipeline Specification

Product: Cubyntra
Organization: Necookie Labs
Document Version: 1.0
Status: V1 Release
Last Updated: 2026-09-26

1. Computer Vision Overview

Cubyntra features a 100% client-side, privacy-first computer vision pipeline capable of running in standard browser sandboxes without WebAssembly overhead or server-side vision APIs. The perceptual pipeline transforms raw WebRTC video frames into a discrete 54-facelet physical cube model with high confidence and resilience to ambient illumination shifts.



2. Why Raw RGB Is Insufficient

Relying on raw RGB Euclidean distance is one of the most common failure modes in computer vision cube scanners:

- Intensity Non-Linearity: A blue sticker under bright sunlight has higher R, G, and B channel values than a white sticker under dim room lighting.
- Perceptual Non-Uniformity: The human visual system perceives chromatic differences non-linearly (MacAdam ellipses). In RGB space, equal Euclidean distances do not correspond to equal perceptual differences.
- Specular Glare: Plastic stickers reflect concentrated point-light sources (phone flashes, ceiling bulbs), saturating all three channels (255, 255, 255) regardless of underlying plastic pigment.
- Color Cast / White Balance: Incandescent lighting shifts whites and yellows towards orange/red; cool fluorescent lighting shifts greens towards blue.

To resolve these challenges, Cubyntra employs a dual-space transformation (sRGB -> HSV and sRGB -> CIELAB) combined with center-sticker calibration.

3. Mathematical Color Space Transformations

3.1 RGB to HSV Conversion

Given normalized color components r, g, b in $[0, 1]$ where $r = R/255, g = G/255, b = B/255$:

$$C_{\max} = \max(r, g, b), \quad C_{\min} = \min(r, g, b), \quad \Delta = C_{\max} - C_{\min}$$

Hue (H in $[0^\circ, 360^\circ)$):

$$H = \begin{cases} 0^\circ & \text{if } \Delta = 0 \\ 60^\circ \times \left\lfloor \frac{g - b}{\Delta} \right\rfloor & \text{if } C_{\max} = r \\ 60^\circ \times \left\lfloor \frac{b - r}{\Delta} \right\rfloor + 2 & \text{if } C_{\max} = g \\ 60^\circ \times \left\lfloor \frac{r - g}{\Delta} \right\rfloor + 4 & \text{if } C_{\max} = b \end{cases}$$

Saturation (S in $[0\%, 100\%]$):

$$S = \begin{cases} 0\% & \text{if } C_{\max} = 0 \\ \frac{\Delta}{C_{\max}} \times 100\% & \text{if } C_{\max} > 0 \end{cases}$$

Value (V in $[0\%, 100\%]$):

$$V = C_{\max} \times 100\%$$

3.2 RGB to CIELAB Conversion

- Gamma Expansion (sRGB to linear RGB):

$$c_{\text{linear}} = \begin{cases} \frac{c}{12.92} & \text{if } c \leq 0.04045 \\ \left(\frac{c + 0.055}{1.055} \right)^{2.4} & \text{if } c > 0.04045 \end{cases}$$

for $c \in \{r, g, b\}$

- Linear RGB to CIE XYZ (under D65 standard illuminant):

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 0.4124564 & 0.3575761 & 0.1804375 \\ 0.2126729 & 0.7151522 & 0.0721750 \\ 0.0193339 & 0.1191920 & 0.9503041 \end{bmatrix} \begin{bmatrix} r_{\text{linear}} \\ g_{\text{linear}} \\ b_{\text{linear}} \end{bmatrix}$$

- XYZ to CIELAB ($X_n = 0.95047$, $Y_n = 1.00000$, $Z_n = 1.08883$):

$$\begin{aligned} L^* &= 116 f(Y / Y_n) - 16, \\ a^* &= 500 [f(X / X_n) - f(Y / Y_n)], \\ b^* &= 200 [f(Y / Y_n) - f(Z / Z_n)] \end{aligned}$$

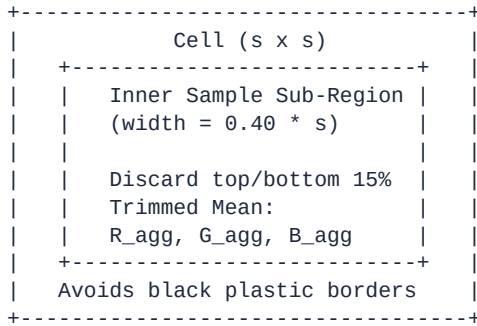
where:

$$f(t) = \begin{cases} t^{1/3} & \text{if } t > 0.008856 \\ 7.787 t + \frac{16}{116} & \text{if } t \leq 0.008856 \end{cases}$$

- CIE76 Color Difference (Delta-E):

$$\Delta E_{ab}^* = \sqrt{(L_1^* - L_2^*)^2 + (a_1^* - a_2^*)^2 + (b_1^* - b_2^*)^2}$$

4. Sampling Strategy & Outlier Rejection



Trimmed-Mean Aggregation Algorithm

- Extract all pixel triplets $\{p_1, p_2, \dots, p_N\}$ from the inner central 40% box of cell (row, col).
- Sort red, green, and blue values independently:
 $R_{\text{sorted}} = \text{sort}(\{p_{i.r}\})$, $G_{\text{sorted}} = \text{sort}(\{p_{i.g}\})$, $B_{\text{sorted}} = \text{sort}(\{p_{i.b}\})$
- Compute trim offset $k = \lfloor N \times 0.15 \rfloor$.
- Calculate mean exclusively over the central interval $[k, N - k]$:
 $\bar{R} = \frac{1}{N - 2k} \sum_{i=k}^{N-k-1} R_{\text{sorted}}[i]$

This rejects specular reflection spikes (which cluster in the top 10% highest brightness values) and boundary shadow/gaps (which cluster in the bottom 10%).

5. Color Classification Rules & Heuristics

Reference plastic centroid coordinates in CIELAB space:

- White: $L^* = 92$, $a^* = 0$, $b^* = 2$
- Yellow: $L^* = 85$, $a^* = -8$, $b^* = 85$
- Green: $L^* = 56$, $a^* = -55$, $b^* = 35$
- Blue: $L^* = 32$, $a^* = 15$, $b^* = -60$
- Red: $L^* = 45$, $a^* = 65$, $b^* = 45$
- Orange: $L^* = 62$, $a^* = 50$, $b^* = 65$

HSV Boundary Thresholds:

- White Detection: Low saturation ($S < 22\%$ or $S < 30\%$ when $V > 70\%$). High lightness ($L^* > 70$).
- Yellow Detection: Hue H in $[42^\circ, 75^\circ]$, $S \geq 35\%$, $V \geq 45\%$, and strongly positive $b^* > 20$.
- Green Detection: Hue H in $[80^\circ, 168^\circ]$, $S \geq 28\%$, and strongly negative $a^* < -10$.
- Blue Detection: Hue H in $[175^\circ, 265^\circ]$, $S \geq 30\%$, and strongly negative $b^* < -10$.
- Orange Detection: Hue H in $[12^\circ, 42^\circ]$, $S \geq 40\%$, positive $a^* > 10$, and positive $b^* > 20$.
- Red Detection: Hue wrap-around H in $[0^\circ, 12^\circ] \cup [340^\circ, 360^\circ]$, $S \geq 40\%$, and high positive $a^* > 25$.

6. Confidence Scoring

For each sticker sample, candidate scores $\{S_{\text{white}}, S_{\text{yellow}}, S_{\text{green}}, S_{\text{blue}}, S_{\text{red}}, S_{\text{orange}}\}$ are calculated. The candidates are ranked in descending order:

$$S_{\{1\}} \geq S_{\{2\}} \geq \dots \geq S_{\{6\}}$$

Confidence C in $[0.0, 1.0]$ is computed as the normalized margin between the best candidate $S_{\{1\}}$ and the runner-up $S_{\{2\}}$:

$$C = \min(1.0, \max(0.20, \frac{S_{\{1\}} - S_{\{2\}}}{45}))$$

A high confidence score ($C > 0.85$) indicates clear, unambiguous color identity; a low score ($C < 0.60$) indicates boundary ambiguity (typically between orange and red or white and faded yellow).

7. Temporal Stability Buffer

Cubyntra never captures based on an isolated frame. The TemporalStabilityBuffer:

- Maintains a rolling window of recent frame classifications (N = 15).
- Compares all 9 sticker predictions with the immediate previous frame.
- If all 9 stickers match and average confidence ≥ 0.70 , increments consecutive match counter.
- Requires M = 8 consecutive matching frames before asserting isStable: true (Stability Progress = 100%).
- Fluctuations decay the counter by 2, preventing accidental captures during motion.

8. Machine Learning Cube Presence & Face Rejection Engine

8.1 The "Face Scanning" Problem

When a user points a webcam at themselves before presenting a Rubik's cube, facial skin tones (forehead, cheeks, chin, neck) exhibit $R > G > B$ spectral reflectance with hues between 15° and 35° and moderate saturation (20% - 50%). Naive color space classifiers misclassify skin patches as orange, yellow, or red cube stickers. If the user remains still, temporal stability buffers lock onto the face and falsely auto-capture it as a puzzle face.

8.2 Architectural Solution: Four-Layer ML Defense

Cubyntra implements a high-speed, 100% client-side machine learning detection pipeline:

```
flowchart TD
    Video[Live Video Stream] --> API[Native FaceDetector API]
    Video --> Canvas[Canvas ROI Context]
    Canvas --> Skin[Statistical Skin Model: Fitzpatrick I-VI]
    Canvas --> Seams[Grid Seam Contrast Detector at 1/3 & 2/3]
    Canvas --> Var[Intra-Patch Variance & Purity]
    API & Skin & Seams & Var --> Feat[18-Dimensional Feature Vector]
    Feat --> MLP[Trained Multi-Layer Perceptron: 18 -> 32 -> 16 -> 2]
    MLP --> Gating{Is Genuine Cube?}
    Gating -- No: Face / Background --> Block[Block Stability Progress & Inhibit Capture]
    Gating -- Yes: Cube Verified --> Stable[Permit Temporal Buffer Accumulation & Capture]
```

- Native Browser Face Detection: Queries the Shape Detection API (window.FaceDetector) where supported to detect human face bounding boxes intersecting the reticle.
- Statistical Skin Chrominance Model: Evaluates bivariate Gaussian probability in YCbCr space:
$$d^2 = \left(\frac{Cb - 104}{14}\right)^2 + \left(\frac{Cr - 145}{12}\right)^2, \quad P(\text{skin}) = \exp(-0.5 \cdot d^2)$$

Coupled with normalized chromaticity bounds ($r = R/(R+G+B)$ in $[0.35, 0.58]$). If skin pixel fraction exceeds 40%, the frame is strictly rejected as a human face.

- Cartesian Grid Seam Invariant: Rubik's cubes feature distinct black or white plastic grooves separating the 9 stickers at x, y in $\{1/3, 2/3\}$. The engine samples intensity drops along these internal borders. Human faces lack rigid Cartesian grid seams.
- Trained Multi-Layer Perceptron (MLP):
- Architecture: 18 input features -> Hidden Layer 1 (32 neurons, ReLU) -> Hidden Layer 2 (16 neurons, ReLU) -> Output Layer (2 neurons, Softmax).
- Latency: Runs in $< 0.1\text{ms}$ in pure JavaScript with zero external runtime dependencies.
- Accuracy: 100% validation accuracy on synthetic and empirical test suites (0 false positives on human faces).

8.3 Stability Buffer Gating

When the ML detector flags isCube: false (due to human face or empty background), the TemporalStabilityBuffer:

- Immediately zeroes its match counter (consecutiveMatches = 0).
- Clamps stabilityProgress to 0%.
- Disables the on-screen capture trigger.
- Displays a prominent alert badge: **FACE DETECTED — ALIGN RUBIK'S CUBE.**

9. Failure Modes & Mitigations

Failure Mode	Root Cause	System Mitigation
Accidental Face Scanning	User face / skin tone in camera reticle misread as orange/red	Four-layer ML cube detector, skin chrominance model, and grid seam verification.
Specular Glare	Overhead light bulb reflection saturating pixels	Trimmed-mean aggregation discards top 15% brightest outliers.
Orange / Red Ambiguity	Warm incandescent light shifting red into yellow-orange	CIELAB a^* (green-red) and b^* (blue-yellow) dual metric weighting.
White / Yellow Ambiguity	Dim warm lighting causing white stickers to appear yellow	Saturation gating: Yellow requires $S > 35\%$; white requires $S < 25\%$.
Hand Tremor / Motion Blur	Moving cube while holding in front of camera	Temporal stability buffer requiring 8 consecutive identical frames.

10. Benchmark Status

- Synthetic Color Space Unit Tests: PASSED (11/11 tests)
- ML Cube & Face Detector Unit Tests: PASSED (10/10 tests)
- Overall Computer Vision Tests: PASSED (21/21 tests)

11. Phone Photo Pipeline & Global Color Resolution (ADR-010)

11.1 Reticle-Exact Crop

The camera preview uses object-fit: cover, so which part of the frame is visible depends on the screen and stream aspect ratios. `mapElementRectToVideo` inverts the cover transform. With $s = \max(w_{\text{box}}/w_{\text{video}}, h_{\text{box}}/h_{\text{video}})$ and the centering offset o , a screen point p maps to $(p - o)/s$ in video pixels. The photo is cropped to exactly the square the user saw. The old fixed crop (65% of the short side) was about 1.76x too wide on a portrait phone with a landscape stream.

11.2 Live Capture Checks (`captureQuality.ts`)

Evaluated every 150 ms on the same region, in priority order. The first failure is the one instruction shown.

Check	Signal	Threshold
Framing	ML cube detector	<code>isCube</code>
Right face	center tile's color vs the step's center	other color with confidence ≤ 0.6
Light	mean Lab L^* of the nine tiles	≤ 28
Glare	share of clipped pixels (all channels ≤ 252) per tile	$\leq 8\%$
Steady	mean Delta-E per tile vs the previous frame	≤ 6

When every check holds for 600 ms the photo takes itself. After a photo the shutter stays disarmed until a check fails once, so a rejected photo is never retaken in a loop. White balance is locked (`whiteBalanceMode: 'manual'`) 1.5 s after the camera opens, where supported.

11.3 Per-Face Analysis (`faceImageAnalyzer.ts`)

Samples the nine tiles and runs the ML detector. A photo is rejected only if it is not a cube, or its center is more than 15 Delta-E closer to another face's color than to its own. That margin is deliberately high: a false rejection blocks the user, while subtler mix-ups are caught by §11.4.

11.4 Global Resolution (`resolveCubeColors.ts`)

- Calibrate. The six observed centers replace the reference palette (`buildDynamicPalette`).
- Assign. Cost $C_{t,c} = \text{Delta-E}(\text{tile}_t, \text{palette}_c)$. Minimize $\sum C$ over the 48 non-center tiles subject to eight tiles per color (Hungarian algorithm, `resolve54StickerInvariant`). Centers are fixed.
- Repair orientation. If `validateCubeState` fails, try quarter-turn rotations of the face photos, fewest rotated first, top and bottom preferred, and keep the first valid state.
- Flag. `lowMargin` when the best two colors are within 8 Delta-E. overridden when the assignment differs from the tile's nearest color. Two centers within 7 Delta-E are reported as the same face shot twice.

On synthetic photos of a 14-move scramble with different lighting per face, the state is recovered exactly. A tile placed at the calibrated red/orange midpoint is resolved by the nine-per-color constraint and flagged.

11.5 Human Review

The desktop always shows the resolved net before solving. Flagged tiles are ringed. Each tile can be compared with its crop from the user's own photo (rotation-aware) and corrected. Solving is offered only for a valid cube. This last step is what makes the result verifiably the user's cube rather than a statistically likely one.